

一、单项选择题

1.B 2.C 3.C 4.A 5.A 6.C 7.D 8.D 9.A 10.A
11.D 12.C 13.C 14.D 15.B

二、判断题

1.√ 2.× 3.× 4.√ 5.× 6.× 7.× 8.× 9.× 10.√ 11.×
12.√ 13.× 14.√ 15.×

三、填空题

1. 操作统一

【注】此空争议有点大，网上书上都没找出答案，最后讨论出了这个还算贴切的答案。

头结点是指向初始地址的一个节点，它本身数据域没有内容，通过它可以标识这个链表，便于对链表进行操作。引入头结，为了使空链表与非空链表处理一致，让对第一个节点的操作和其他节点的操作一致，保证了处理第一个节点和后面的节点设计的算法相同，实现程序的高效性。

2. 假溢出

3. 逻辑结构

4. 顺序

5. 最小生成树、 $O(n \log n)$

6. 队列

7. $\log_2(n+1)-1$

8. $n-i$ 、 $n-i-1$

9 . 略 (这题应该是有问题)

10 . 4

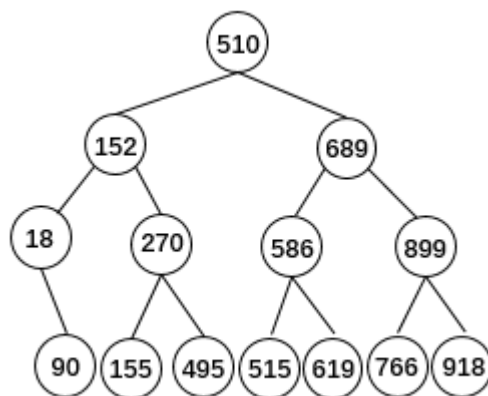
13 . $n/2$ 次向下、 $n-1$ 次重建堆

【注】此题答案也不确定，不可全信

14 . 出度

四、简答题

1 .



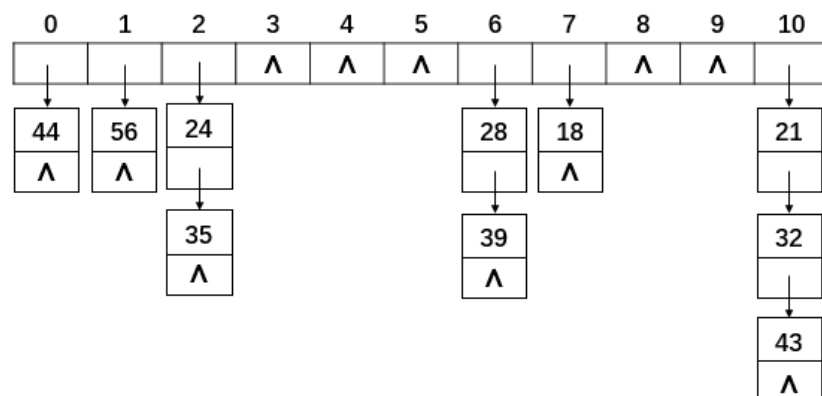
$$ASL_{\text{成功}} = (1+2+3+4+3+4+4+2+3+4+4+3+4+4)/14 = 45/14$$

$$ASL_{\text{失败}} = (3+4+4+4+4+4+4+4+4+4+4+4+4+4)/15 = 59/15$$

2 . 先计算出各关键字的散列地址

关键字	21	18	28	24	32	56	43	35	39	44
地址	10	7	6	2	10	1	10	2	6	0

①由此构造出以链地址法解决冲突的散列表为：



$$ASL_{成功} = (1+1+1+2+1+2+1+1+2+3)/10 = 15/10$$

$$ASL_{失败} = (1+1+2+2+1+3)/11 = 10/11$$

②线性探测法处理冲突的散列表为

地址	0	1	2	3	4	5	6	7	8	9	10	11
关键字	43	56	24	25	44		28	18	39		21	32

$$ASL_{成功} = (3+1+1+2+5+0+1+1+3+1)/10 = 18/10$$

$$ASL_{失败} = (6+5+4+3+2+1+4+3+2+1+8)/11 = 39/11$$

【注】在写此答案时，突然有点怀疑它的正确性，但也没发现问题，望注意

(3)

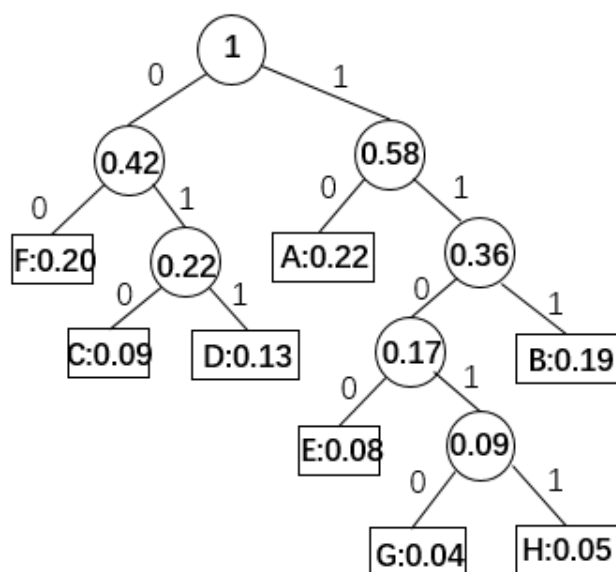
```

1  typedef struct tagNode           // 链表结点
2  {
3      int data;                    // 关键字
4      struct tagNode *next;        // 下一个结点
5  }Node, *PNode;
6
7
8  typedef struct tagHashTable // 哈希表结点
9  {
10     union{
11         int index;
12         int nHashValue;
13     }INDEX
14     Node* firstchild;
15     struct tagHashTable *next;
16 }HashTable, *PHashTable;

```

【注】此问也是网上找了好久才找到个这，以本人的水平，判断不了对错，仅供参考。

3 .



哈夫曼编码

H	11011
G	11010
E	1100
B	111
D	011
C	010
A	10
F	00

4 .

深度优先: A B C F E D

广度优先: A B E C D F

	A	B	C	D	E	F
A	0	1	∞	∞	1	∞
B	1	0	1	1	∞	∞
C	∞	1	0	∞	∞	1
D	∞	1	∞	0	1	∞
E	1	∞	∞	1	0	1
F	∞	∞	1	∞	1	0

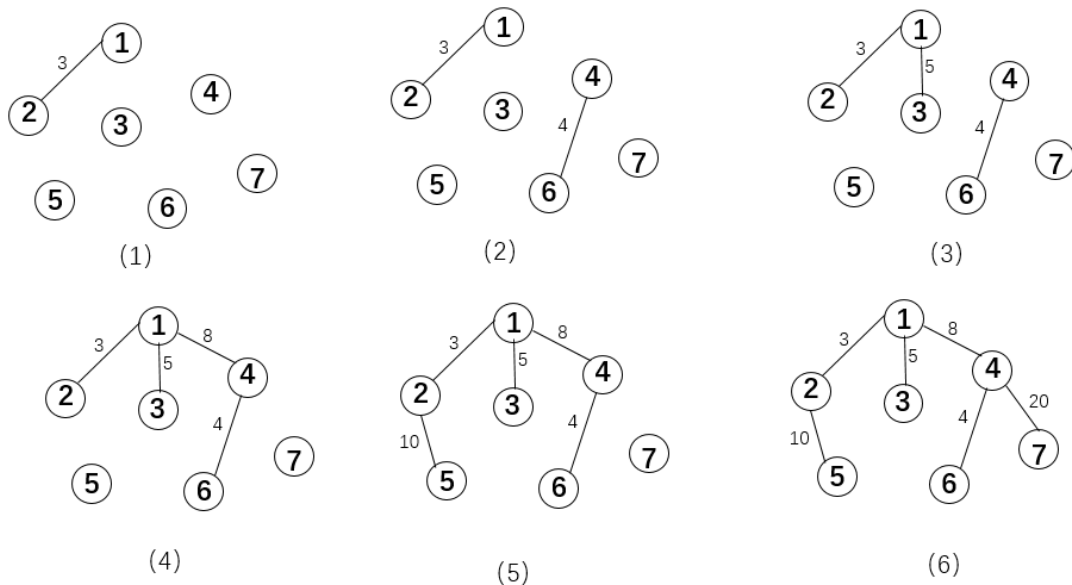
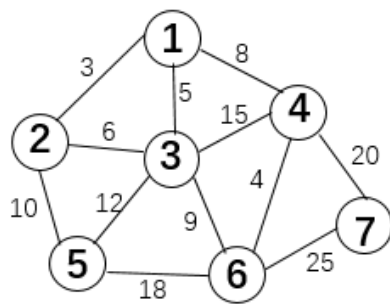
5 . (1) 生成树是一个包含 n 个结点的连通图 G 的一个连通子图, 该子图必须包含 G 中的所有 n 个结点以及 G 中的 $n-1$ 条边。

最小生成树是 G 的所有可能的生成树中, $n-1$ 条边的权值总和最小的那一个 (或多个) 生成树。

(2) 克鲁斯卡尔算法思想: 先构造一个只包含 n 个顶点而边集为空的子图, 把子图中各个顶点看成各棵树上的根结点, 然后从图的边集 E 中选取一条权值最小的边, 若该边的两个顶点分属不同的树, 则将其加入子图, 即把两棵树合成一棵, 反之, 若该条边的两个顶点已落在同一棵树上, 则跳过, 取下一条权值最小的边, 依此类推, 直到

森林中合并成了一棵树，也即图中含有 $n-1$ 条边为止。

(3)



五、算法设计题

```

1. 1) 1 void Del_x(LinkList &L, int x)
      2 {
      3     LNode *p=L->next, *pre=L, *q;
      4     while(p != NULL)
      5     {
      6         if(p->data == x)
      7         {
      8             q = p;
      9             p = p->next;
          10             pre->next = p;
          11             free(q);
          12         }
          13         else
          14         {
          15             pre = p;
          16             p = p->next;
          17         }
          18     }
          19 }
          20

```

```

2) 1 void Insert_x(LinkList &L, int x) //尾插法
    2 {
    3     Lnode *p, *m;
    4     m = (Lnode*)malloc(sizeof(Lnode));
    5     m->next = NULL;
    6     m->data = x;
    7     p = L;
    8     while(p->next != NULL)
    9         p = p->next;
   10     p->next = m;
   11 }

```

2. 1)

```

1 int Out_degree(MGraph G, VertexType i)
2 {
3     int j, sum=0;
4     for(j=0; j<G.vernum; ++j)
5     {
6         if(AdjMatrix[i][j]==1)
7             sum++;
8     }
9     return sum;
10 }

```

出度和入度基本一样，换成 AdjMatrix[j][i] 就好。

3.

```

1 void QuickSort(Sqlist &L, int low, int high)
2 //对从L.r[low]到L.r[high]的关键字进行排序
3 {
4     int temp;
5     int i=low, j=high;
6     if(low < high)
7     {
8         temp = L.r[low];
9         //下面这个循环完成了一趟排序，即将数组中大于 K 的关键字放在左边，
10        //小于 K 的关键字放在右边
11        while(i<j)
12        {
13            while(i<j && L.r[j].key<=temp.key) --j;
14            if(i<j)
15            {
16                L.r[i] = L.r[j];
17                ++i;
18            }

```

```
18 | }
19 | while(i<j && L.r[j].key>temp.key) ++i;
20 | if(i<j)
21 | {
22 |     L.r[j] = L.r[i];
23 |     --j;
24 | }
25 | }
26 | }
27 | L.r[i] = temp;
28 | QuickSort(L, low, i-1);
29 | QuickSort(L, i+1, high);
30 | }
31 |
```